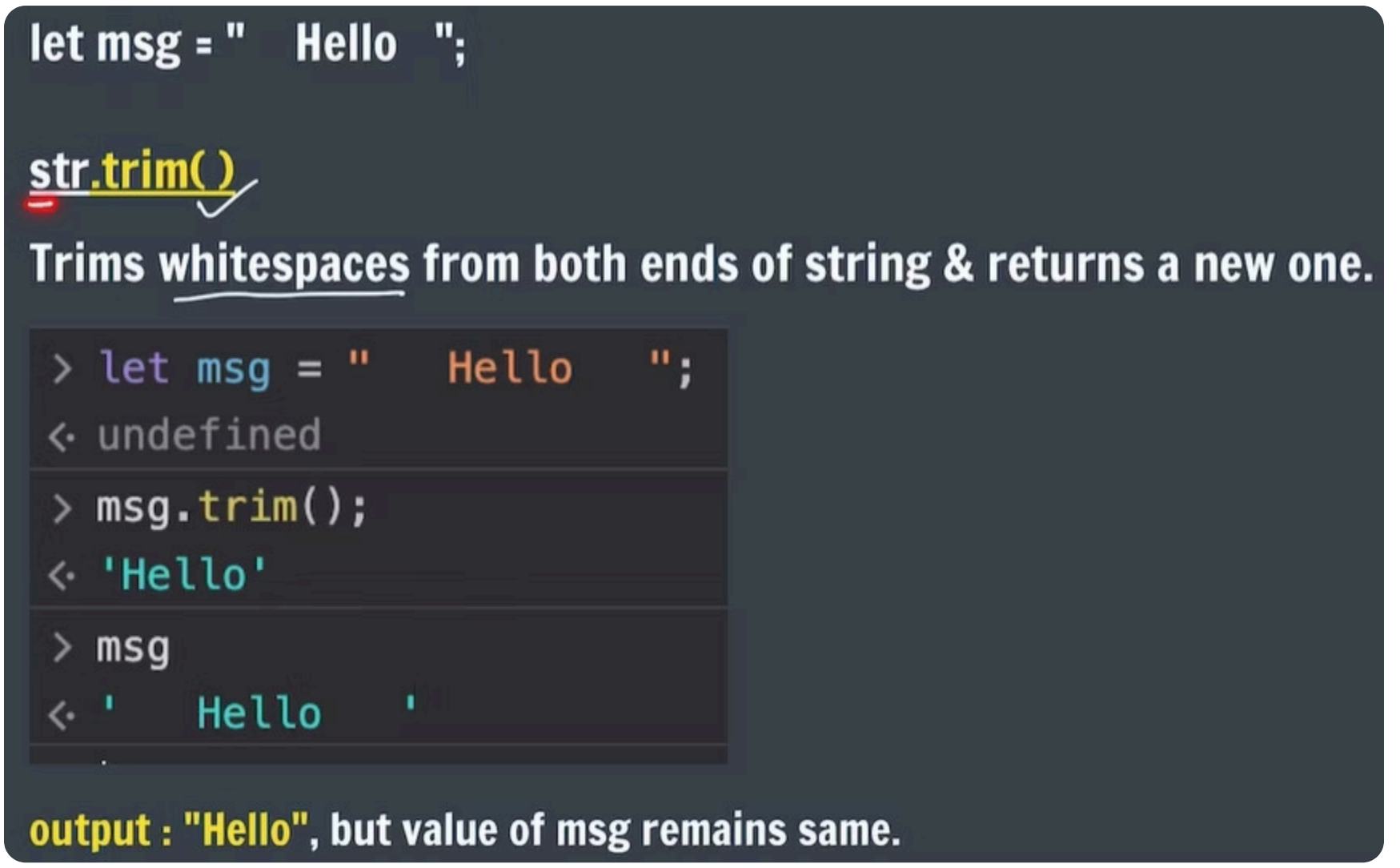
String Methods



String methods are built-in actions you can perform on string variables (objects). Think of them as powerful tools that let you manipulate, analyze, and transform text data with just a single line of code.

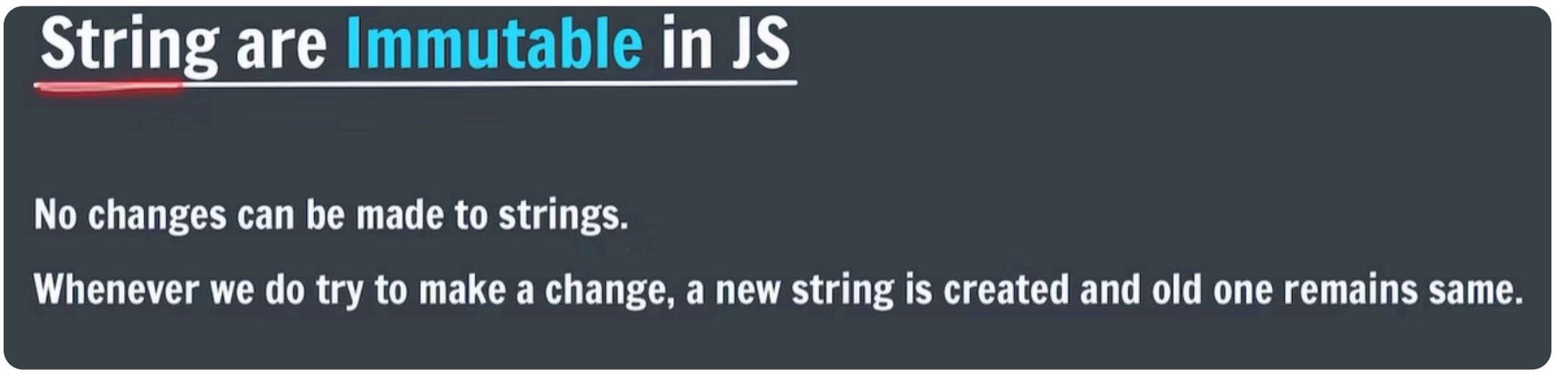
Basic Format

Every string method follows a consistent syntax pattern:

```
stringVariable.method()
```

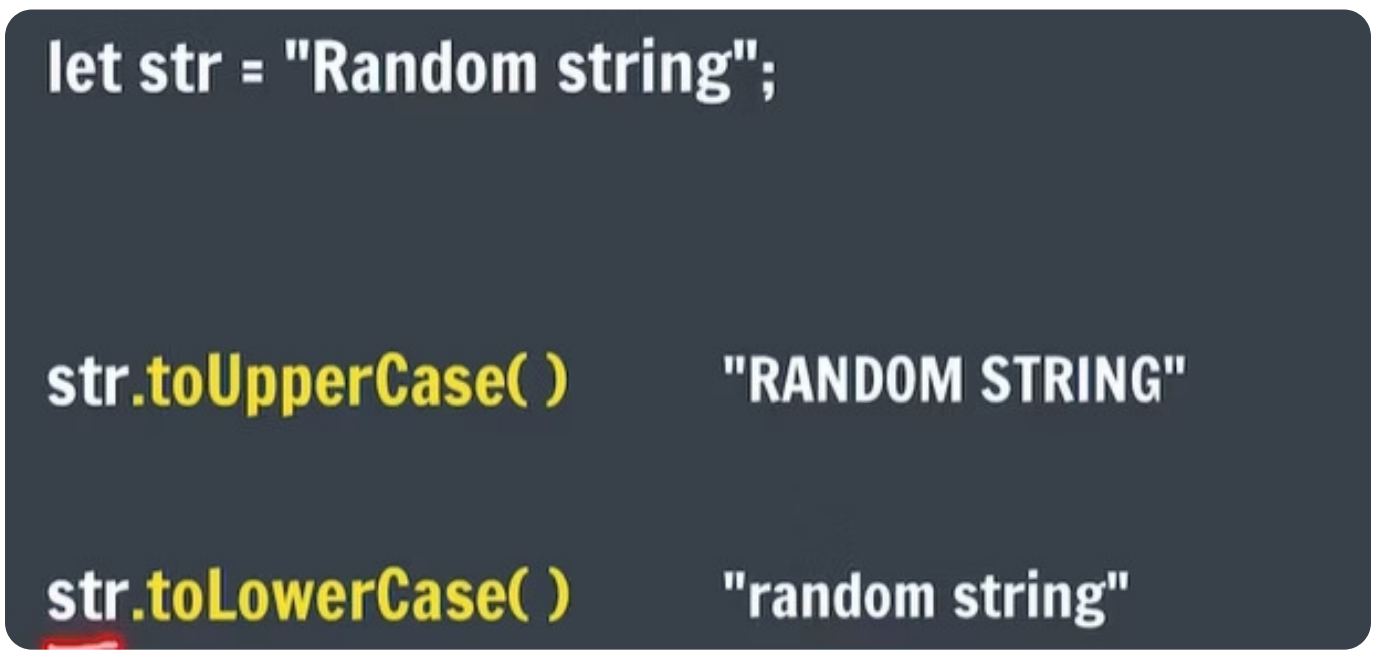
How It Works

Methods are called using dot notation. The string variable comes first, followed by a period and the method name with parentheses.



Pro Tip

String methods don't modify the original string—they return a new value. Remember to store the result if you need it later!



String Methods with Arguments

Argument is a some value that we pass to the method.

Format

```
stringName.method( arg )
```

indexOf

Returns the first index of occurrence of some value in string. Or gives -1 if not found.

```
let str = "IloveCoding";
```

0 1 2 3 4 . . . .

```
let str = "IloveCoding";
```

0 1 2 3 4 . . . .

```
str.indexOf("love")      // 1

str.indexOf("J")         // -1 (not found)

str.indexOf("o")         // 2 (only 1 index)
```

Method Chaining

Using one method after another. Order of execution will be left to right.

```
str.toUpperCase( ).trim( )
```

```
let msg = "    hello   ";
// let newMsg = msg.trim();
// console.log("after trim : ", newMsg);
// newMsg = newMsg.toUpperCase();
// console.log("after uppercase : ", newMsg);
let newMsg = msg.trim().toUpperCase();
```

Slice Method

Returns a part of the original string as a new string.

```
let str = "IloveCoding";

    str.slice(5)                // "Coding"

                5
    str.slice(1, 4)             // "love"

    str.slice(-num)   =   str.slice(length-num)
```

Replace and Repeat

Searches a value in the string & returns a new string with the value replaced

```
let str = "IloveCoding";

    str.replace("love", "do")    // "IdoCoding"

    str.replace("o", "x")        // "IlxveCoding"
```

Returns a string with the number of copies of a string

```
let str = "Mango";

    str.repeat(3)                // "MangoMangoMango"
```

Practice Question

Qs. For the given string:

let msg = " help!";

Trim it & convert it to uppercase.

Qs. For the String -> `let name = "Opq Technology"`, predict the output for following :

`name.slice(4, 9)`

`name.indexOf("no")`

`name.replace("Tech", "Comp")`

Qs. Separate the **"`Technology`"** part in above string & replace **"T"** with **"t"** in it.

Arrays

What if you need to store marks of 100 students?

The Inefficient Way: Individual Variables

```
let mark1 = 95;
let mark2 = 88;
let mark3 = 92;
// ... (imagine 97 more variables)
```

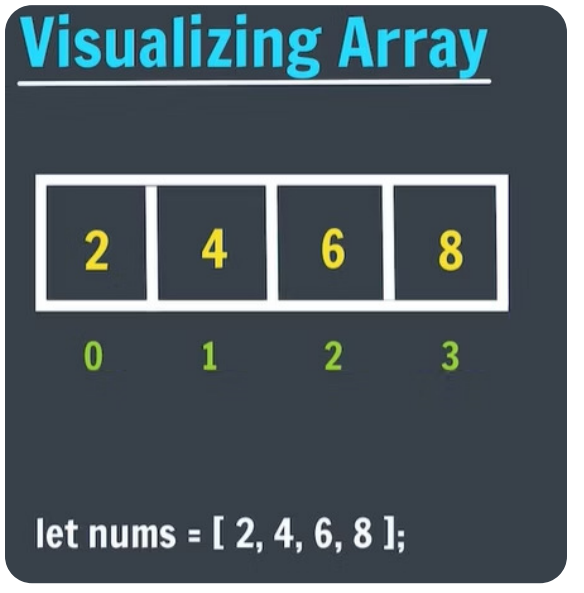
The Efficient Way: Using an Array

```
let marks = [95, 88, 92, ...];

// Accessing data:
// console.log(marks[0]); // Outputs 95

// Adding a new mark:
// marks.push(85);
```

Arrays let you store multiple values in one variable.



Why Arrays are useful

- Store multiple values together(useful in storing related values together.)
- Easy to access and update
- Can grow or shrink as needed

Creating Arrays

```
let marks = [99, 85, 93, 76, 62];
let names = ["adam", "bob", "catlyn"];
let info = ["aman", 25, 6.1];   //mixed array

//empty array
let newArr = [];
```

arr.length

Arrays are Mutable

```
> let fruits = ["mango", "apple", "litchi"];
<· undefined
> fruits[0] = "banana";
<· 'banana'
> fruits
<· ▶ (3) ['banana', 'apple', 'litchi']
```

Array Methods

Push : add to end

Unshift : add to start

Pop : delete from end & returns it

Shift : delete from start & returns it

Practice Qs

Qs. For the given start state of an array, change it to final form using methods.

start : `['january', 'july', 'march', 'august']`

final : `['july', 'june', 'march', 'august']`

Array Methods

indexOf : returns index of something

```
> primary.indexOf("yellow");
<· 1
> primary.indexOf("green");
<· -1
> primary.indexOf("Yellow");
<· -1
```

includes : search for a value

```
> primary.includes("red");
<· true
> primary.includes("green");
<· false
```

```
> let primary = ["red", "yellow", "blue"];
```

0 1 2

```
> cars
<· ▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.indexOf("bmw");
<· 1
> cars.indexOf("xuv");
<· 2
> cars.indexOf("XUV");
<· -1
> let marks = [99, 89, 67, 42, 100];
<· undefined
> marks.indexOf(100);
<· 4
> marks.indexOf(97);
<· -1
```

```
▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
cars.includes("audi");
true
cars.includes("toyota");
false
```

Array Methods

```
> let primary = ["red", "yellow", "blue"];
<· undefined
> let secondary = ["orange", "green", "violet"];
<· undefined
```

concat : merge 2 arrays → concatenate

```
> primary.concat(secondary);
<· ▶ (6) ['red', 'yellow', 'blue', 'orange', 'green', 'violet']
```

reverse : reverse an array

```
> primary.reverse();
<· ▶ (3) ['blue', 'yellow', 'red']
```

slice : copies a portion of an array

```
> let colors = ["red", "yellow", "blue", "orange", "pink", "white"
```

```
> colors.slice()
<· ▶ (6) ['red', 'yellow', 'blue', 'orange', 'pink', 'white']
> colors.slice(2);
<· ▶ (4) ['blue', 'orange', 'pink', 'white']
> colors.slice(2, 3);
<· ▶ ['blue']
> colors.slice(-2);
<· ▶ (2) ['pink', 'white']
```

```
> cars
<· ▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.slice(-1)
<· ▶ ['maruti']
> cars.slice(-2)
<· ▶ (2) ['xuv', 'maruti']
> cars.slice(-3)
<· ▶ (3) ['bmw', 'xuv', 'maruti']
> cars.slice(-4)
<· ▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
> cars.slice(-5)
<· ▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
```

splice : removes / replaces / add elements in place

splice(start, deleteCount, item0...itemN)

↑ OP

```
> colors.splice(4);
<· ▶ (2) ['pink', 'white']
> colors
<· ▶ (4) ['red', 'yellow', 'blue', 'orange']
> colors.splice(0, 1);
<· ▶ ['red']
> colors
<· ▶ (3) ['yellow', 'blue', 'orange']
> colors.splice(0, 1, "black", "grey");
<· ▶ ['yellow']
> colors
<· ▶ (4) ['black', 'grey', 'blue', 'orange']
```

```
cars
▶ (4) ['audi', 'bmw', 'xuv', 'maruti']
cars.splice(3);
▶ ['maruti']
cars
▶ (3) ['audi', 'bmw', 'xuv']
cars.splice(0,1);
▶ ['audi']
cars
▶ (2) ['bmw', 'xuv']
cars.push("maruti");
cars.push("ferrari");
4
cars
▶ (4) ['bmw', 'xuv', 'maruti', 'ferrari']
cars.splice(1, 2);
▶ (2) ['xuv', 'maruti']
cars
▶ (2) ['bmw', 'ferrari']
cars.splice(0, 0, "toyota", "xuv", "maruti");
▶ []
cars
▶ (5) ['toyota', 'xuv', 'maruti', 'bmw', 'ferrari']
```

```
cars
▶ (5) ['toyota', 'xuv', 'maruti', 'bmw', 'ferrari']
cars.splice(1, 0, "mercedes");
▶ []
cars
▶ (6) ['toyota', 'mercedes', 'xuv', 'maruti', 'bmw', 'ferrari']
cars.splice(1, 1, "gwagon");
▶ ['mercedes']
cars
▶ (6) ['toyota', 'gwagon', 'xuv', 'maruti', 'bmw', 'ferrari']
```

sort : sorts an array

```
> let days = ["monday", "sunday", "wednesday", "tuesday"];
<· undefined
> days.sort();
<· ▶ (4) ['monday', 'sunday', 'tuesday', 'wednesday']
```

```
> let squares = [25, 16, 4, 49, 36, 9]
<· undefined
> squares.sort();
<· ▶ (6) [16, 25, 36, 4, 49, 9]
```

Practice Qs

Qs. For the given start state of an array, change it to final form using splice.

start : `['january', 'july', 'march', 'august']`

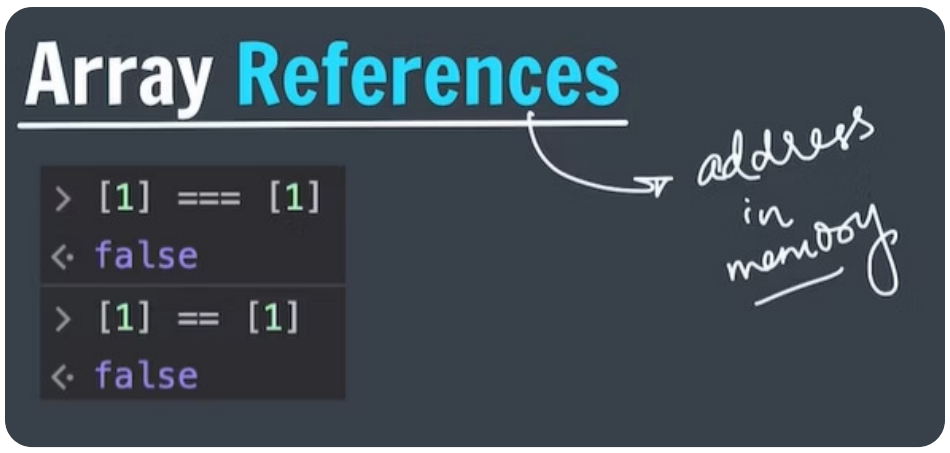
final : `['july', 'june', 'march', 'august']`

Qs. Return the index of the "javascript" from the given array, if it was reversed.

```
['c', 'c++', 'html', 'javascript', 'python', 'java', 'c#', 'sql']
```

```
let months = ["january", "july", "march", "august"];
undefined
months.splice(0, 2, "july", "june");
▶ (2) ['january', 'july']
mont
▶ Uncaught ReferenceError: mont is not defined
    at <anonymous>:1:1
months
▶ (4) ['july', 'june', 'march', 'august']
```

```
lang
▶ (8) ['c', 'c++', 'html', 'javascript', 'python', 'java', 'c#', 'sql']
lang.reverse().indexOf("javascript");
```



1) Primitives: compare the *value*

```js
"hi" === "hi"     // true
10 === 10         // true
true === true     // true
```

```
> "name" == "name"
true
> "name" === "name"
true
> [1] === [1]
false
> [1] == [1]
false
> [] == []
false
> [] === []
false
```

2) Objects: compare the *reference*, not the content

```js
{a:1} === {a:1}  // false  (different objects in memory)
```

Even arrays:

```js
[1,2] === [1,2]  // false
```

```
> let arr = ['a', 'b'];
undefined
> let arrCopy = arr;
undefined
> arrCopy
▶ (2) ['a', 'b']
> arrCopy.push('c');
3
> arr
▶ (3) ['a', 'b', 'c']
```

```
> arr == arrCopy
true
```

Constant Arrays

```
const arr = [1, 2, 3, 4];
```

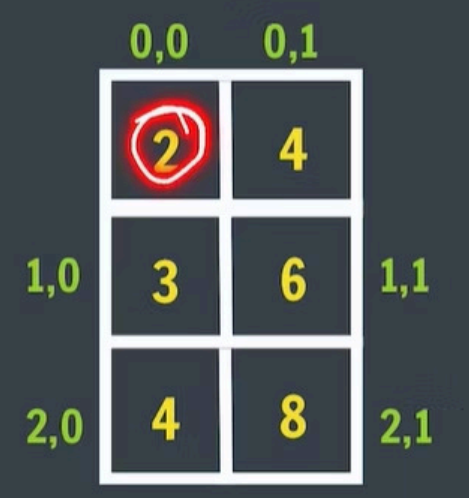
```
> const arr = [1, 2, 3];
<· undefined
> arr
<· ▶ (3) [1, 2, 3]
> arr.push(4);
<· 4
> arr
<· ▶ (4) [1, 2, 3, 4]
> arr.pop();
<· 4
> arr
<· ▶ (3) [1, 2, 3]
> arr = [1, 2, 3];
✕ ▶ Uncaught TypeError: Assignment to constant variable.
    at <anonymous>:1:5
>
```

Nested Arrays → multi di

array of arrays

```
> let nums = [ [2, 4], [3, 6], [4, 8] ];
<· undefined
> nums
<· ▼ (3) [Array(2), Array(2), Array(2)]
    ▶ 0: (2) [2, 4]
    ▶ 1: (2) [3, 6]
    ▶ 2: (2) [4, 8]
      length: 3
    ▶ [[Prototype]]: Array(0)
```

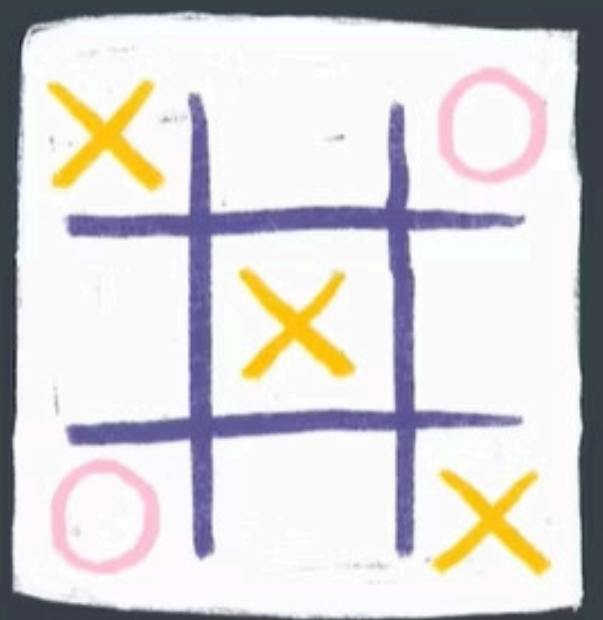
```
> let nums = [ [2, 4], [3, 6], [4, 8] ];
```



rows
cols

nums[0][0] //2

```
> let nums = [ [1, 2], [3, 4], [4, 5] ];
<· undefined
> let nums = [ [2, 4], [3, 6], [4, 8] ];
<· undefined
> nums
<· ▼ (3) [Array(2), Array(2), Array(2)]
    ▶ 0: (2) [2, 4]
    ▶ 1: (2) [3, 6]
    ▶ 2: (2) [4, 8]
      length: 3
    ▶ [[Prototype]]: Array(0)
> nums.length
<· 3
```

Practice Qs

Qs. Create a nested array to show the following tic-tac-toe game state.